COP 3502C Programming Assignment # 3

Linked Lists, Queues, and Stacks

Please read all the pages before starting to write your code

Please include the following commented lines at the beginning of your code to declare your authorship of the code:

/* COP 3502C PA3

This program is written by: Your Full Name */

Compliance with Rules: *The UCF Golden Rules apply to this assignment and its submission. In addition, all assignment policies outlined in the syllabus also apply here. The instructor or TA may ask any student to explain any part of their code to verify authorship and ensure clarity. Sharing this assignment description or your code, in whole or in part, with anyone or anywhere is a violation of course policy. Likewise, using code obtained from any outside source, including AI tools, will be considered cheating. Your primary resources should be class notes, lectures, labs, and TAs. These rules are in place to ensure fairness, maintain academic integrity, and give every student the opportunity to develop their own problem-solving and coding skills.*

Deadline:

Please check Webcourses for the official assignment deadline. Assignments submitted by email will not be graded, and such emails will not receive a response in accordance with course policy.

What to do if you need clarification on the problem?

If you need clarification on the problem, a discussion thread will be created on Webcourses, and you are strongly encouraged to post your questions there. You may also ask questions on Discord; if TAs are available, they can respond, and other students might also contribute, helping you get answers more quickly. For further clarification on the requirements, you are welcome to visit the TAs or the instructor during office hours.

How to get help if you're stuck?

As stated in the course policy, office hours are the primary place to seek help. While we may occasionally respond to emails, debugging requests are best addressed during office hours rather than through email.

PA3: The Whisker Wellness Clinic

Objective:

In this assignment, you will:

- Practice implementing and manipulating singly linked lists to model a dynamic waiting queue
- Using stack data structure operations
- Simulate a discrete-time system involving two service providers
- Work with **structs, pointers, and pointer reassignment**

Background story:

The Whisker Wellness Clinic operates for exactly 8 hours (480 minutes) each day. Two veterinarians, Dr. Uno and Dr. Dos, are available throughout the clinic day. Cats arrive throughout the day and wait in a shared queue to receive treatment. Unfortunately, a chemical leak was discovered in Dr. Dos's treatment room. As a result, the clinic must conduct contact tracing to notify the owners of all cats treated by Dr. Dos.

However, the clinic's visit log for that day became corrupted. Arrival times were originally listed in chronological order, but due to corruption, this ordering is no longer guaranteed. Your task is to reconstruct the events of the day and determine:

- Which cats were successfully treated
- Which cats could not be accommodated
- Which cats were treated by Dr. Dos (for contact tracing)

Required Structures:

```
typedef struct Cat_s {
    int arrival;              // Minutes since clinic opened (Unique)
    char *name;               // Unique cat name
    int duration;             // Required treatment time in minutes
} Cat;

typedef struct SLLNode_s {
    Cat *cat;                 // Pointer to a cat
    struct SLLNode_s *next;
} SLLNode;
```

Simulation Rules:

Doctor Assignment Rules

- Both doctors are available at time 0.
- If a doctor is free **and** the next cat in the queue has already arrived:
    - Assign the cat immediately.
    - Treatment begins at that minute.
    - The doctor becomes busy for `duration` minutes.

Priority Rule

All cats wait in a single shared queue. The queue follows a strict FIFO (First-In, First-Out) policy, so cats are treated in the exact order they arrive. If both doctors become available at the same time, the next waiting cat must be assigned to Dr. Uno first.

Time Constraint

A cat can be accommodated only if its entire treatment is completed before the clinic closes. Specifically, the treatment must begin before minute 480 and must finish at or before minute 480.

Formally:

A cat can be treated only if `start_time + duration ≤ 480`

Exposure Rule
A cat is considered exposed if it was treated by Dr. Dos at any time during the day.

Zero-Duration Rule
If a cat's duration is 0, this means the cat left the clinic immediately without receiving any treatment. Such a cat does not occupy a doctor's time and is not treated. This cat should not appear in the treatment log, accommodation list, or exposure report.

Input:

The program reads an arbitrary number of input lines in the following format:

```
<arrival_time> <name> <duration>
```

Where:

- arrival_time is a non-negative integer representing the minute the cat arrived.
- name is a string (maximum length of 25 characters) representing the cat's name. The name must be **dynamically allocated**.
- duration is a non-negative integer representing the required treatment time (in minutes).
- Arrival times are guaranteed to be unique. In other words, no two cats will arrive at the clinic at the same time.
- Cat names are guaranteed to be unique.

Input terminates when a single line containing only a `-1` is encountered.

Output:

The program must produce output in the following format. For each successful treatment (in chronological simulation order), print:

```
Doctor X treated <name> at <time>
```

Where:

- X is either **Uno** or **Dos**
- <time> is the minute the treatment began

After printing all successful treatments, print any cats that could not be accommodated (if any) in the following format:

```
Cannot accommodate <name>
```

Print rejected cats in the chronological order in which they are determined to be unable to receive treatment during the simulation (i.e., cannot accommodate due to the clinic closing).

Finally, print the exposure report:

If one or more cats were treated by **Dr. Dos**, print the names of exposed cats in **reverse chronological order** of treatment (most recently treated first).

```
Exposed Cats
<name>
<name>
...
```

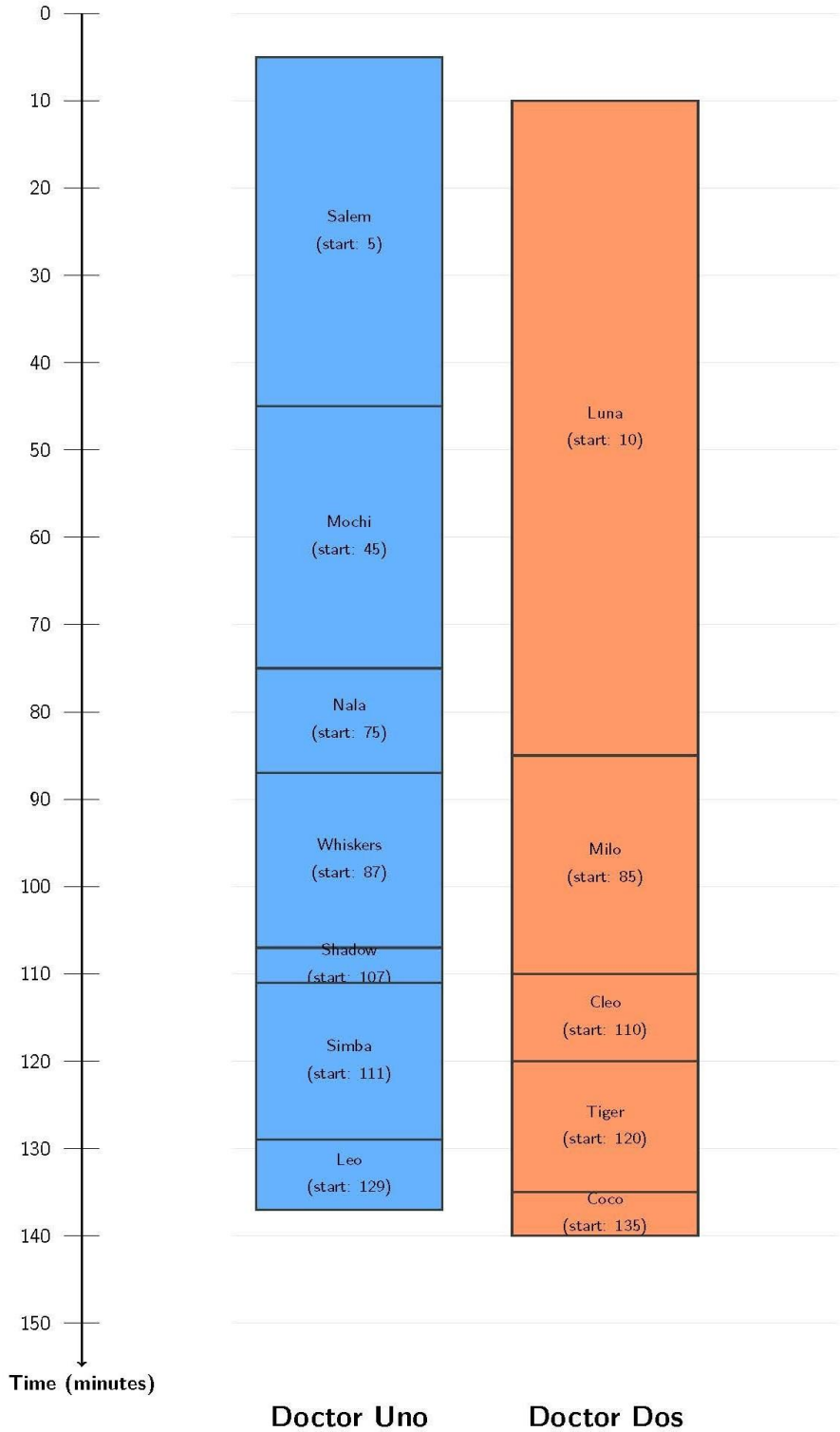
If no cats were treated by Dr. Dos, print:

```
No Exposed Cats
```

Sample Program Run:

Example Input (standard input)	Example Output (standard output)
25 Whiskers 20 10 Luna 75 60 Tiger 15 11 Mochi 30 33 Shadow 4 14 Puff 0 5 Salem 40 44 Cleo 10 18 Milo 25 70 Leo 8 13 Nala 12 52 Simba 18 90 Coco 5 475 Midnight 10 -1	Doctor Uno treated Salem at 5 Doctor Dos treated Luna at 10 Doctor Uno treated Mochi at 45 Doctor Uno treated Nala at 75 Doctor Dos treated Milo at 85 Doctor Uno treated Whiskers at 87 Doctor Uno treated Shadow at 107 Doctor Dos treated Cleo at 110 Doctor Uno treated Simba at 111 Doctor Dos treated Tiger at 120 Doctor Uno treated Leo at 129 Doctor Dos treated Coco at 135 Cannot accommodate Midnight Exposed Cats Coco Tiger Cleo Milo Luna

Refer to the image below for a partial visualization of the expected output. It displays only the cats that received treatment. The vertical line represents the elapsed time since the clinic opened. In the full output, this line should extend to 480 minutes; however, for brevity, the figure shown here displays only up to 150 minutes.



Implementation Requirements:

1. Your solution must use a linked list.
2. Implement the queue using a linked list and include standard queue operations such as create/initialize, enqueue, dequeue, peek, and empty.
3. Although the queue is for cats, the arrival times in the input are not ordered due to corrupted logs. Therefore, when inserting cats into the queue, you must combine the enqueue operation with sorted linked list insertion. While doing this, correctly update both the front and back pointers after each insertion, when needed. You may not use a separate loop solely to update the back pointer.
4. You must implement and use a linked list-based stack to store and print the exposed cats. The stack must include standard operations such as create/initialize, push, pop, peek, and empty.
5. You are not allowed to use any sorting algorithm or built-in sort function. Using such methods will result in a -100 penalty.
6. Do not use any global variables.
7. You must free all dynamically allocated memory. Your program must have zero memory leaks to receive full credit.
8. Use standard input/output (scanf/printf). File I/O is not allowed.
9. For strings, first read the input into a statically declared char array. Then allocate the exact required memory based on the string length and copy it using strcpy, as demonstrated in class. Any other method will result in a -100 penalty.
10. Your code must compile and run successfully on CodeGrade to receive any credit.
11. Submit well-structured and well-commented code to avoid penalties.

Hints and Recommended Approach (Important):

- When reconstructing the chronological order of the queue, perform ordered insertion at the time of insertion. You are not allowed to sort the list after it has been built. Each new node must be placed immediately into its proper position according to arrival time.
- Define appropriate structures to help you define the queue and the stack.
- Consider writing helper functions to manage the queue and the stack.
- Test with small inputs before trying larger cases
- While processing the queue of cats and running the simulation:
    - Track two timers, one for each doctor
    - Based on the arrival time, duration, and current timer for the doctor, assign the cat to the appropriate doctor and update their timer

Deliverables:

- Please submit the source file, `main.c`, via Webcourses->Codegrade.